



Inżynieria Oprogramowania

dr Katarzyna Grzesiak-Kopeć





02

Najlepsze praktyki

6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami



6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami



Technika wodospadu

01 Analiza wymagań

02 Projektowanie

03 Kodowanie i testowanie modułów

04 Integracja podsystemów

05 Testowanie systemu



Opóźnia identyfikację ryzyka



Trudno ocenić rzeczywisty czas do ukończenia projektu



Odsuwa w czasie integrację i agreguje testowanie

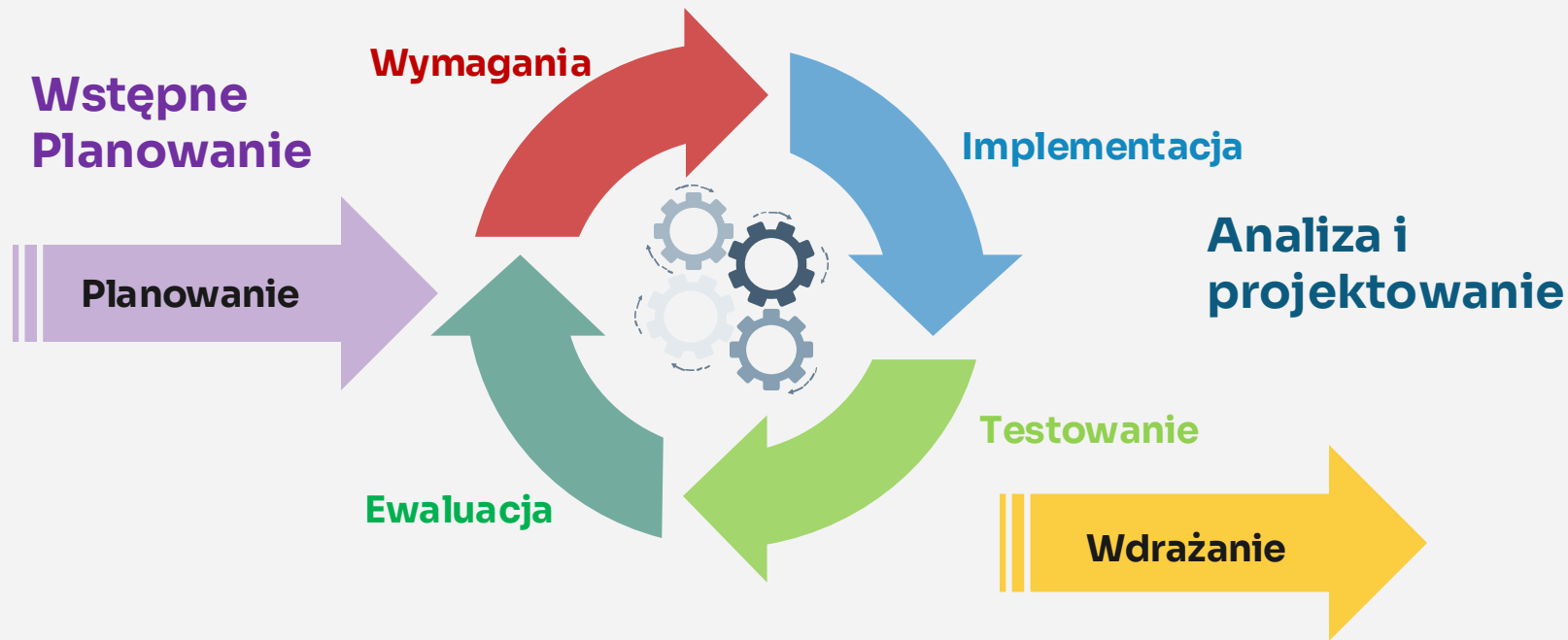


Wyklucza wczesne wdrażanie



Częste niezaplanowane iteracje

Inżynieria oprogramowania



W wyniku każdej iteracji powstaje wykonywalny moduł

Zalety techniki iteracyjnej

- ✓ Rozwiązanie ryzykownych kwestii zanim będą duże inwestycje
- ✓ Umożliwia wczesną ocenę przez klienta
- ✓ Ciągłe integrowanie i testowanie
- ✓ Realizacja zadania w małych, krótkoterminowych krokach
- ✓ Umożliwia częściowe wdrażanie produktu

Scrum, Kanban, Lean Development, Integracja z DevOps



6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami



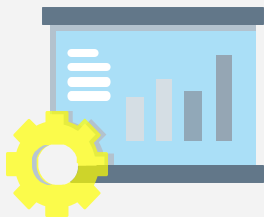
Zarządzanie wymaganiami

Należy upewnić się, że:



- ✓ rozwiązujemy właściwy problem
- ✓ budujemy właściwy system

Po przez systematyczne:



- ✓ gromadzenie,
- ✓ organizację,
- ✓ dokumentowanie,
- ✓ zarządzanie,
- ✓ zmieniającymi się wymaganiami.

Zarządzanie wymaganiami



Analiza problemu

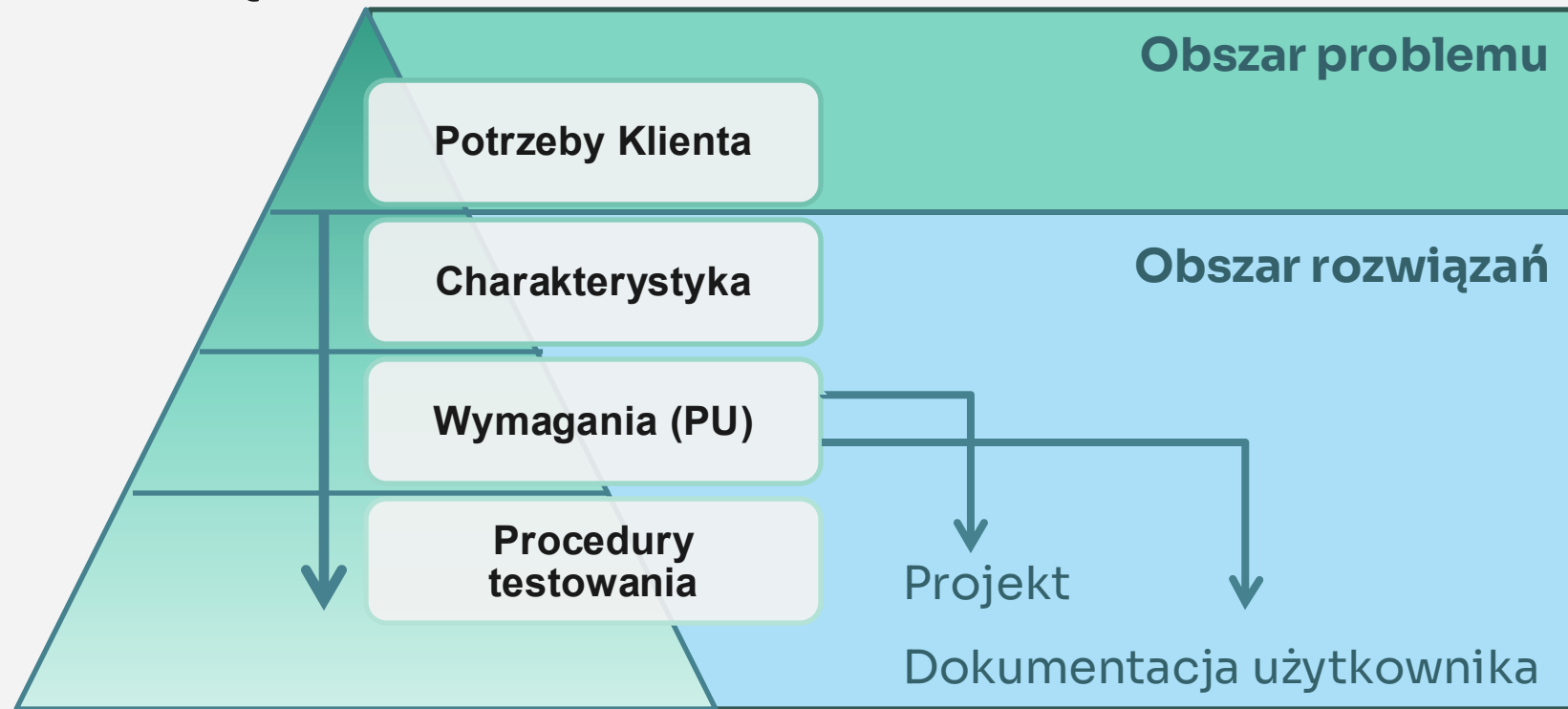
Zrozumienie potrzeb klienta

Ogarnięcie całości

Ciągła weryfikacja definicji systemu

Budowa właściwego systemu

Zarządzanie wymaganiami



Korzyści ze śledzenia wymagań



Ocena wpływu zmiany wymagania

Ocena stopnia realizacji wymagania

Ocena stopnia realizacji całego projektu

Czy wszystkie wymagania zostały zrealizowane?

Czy aplikacja robi tylko to, co miała robić?

Zarządzać zmianami

Nowoczesne podejście

Użytkownik wybaczy brak funkcji,
ale nie wybaczy aplikacji, która
działa wolno lub jest niebezpieczna



User stories, epiki, backlogi produktowe



Narzędzia: Jira, Azure DevOps, Trello



Zarządzanie wymaganiami niefunkcjonalnymi:
wydajność, bezpieczeństwo, UX



Śledzenie wymagań



Priorytety według wartości biznesowej

6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami



Co to jest architektura?



Zbiór zasad, decyzji, reguł lub wzorców dla dowolnego systemu, który zapobiega niepotrzebnej radosnej twórczości projektantów.

Określa najważniejsze elementy organizacji systemu:

- ✓ Elementy strukturalne i ich interfejsy
- ✓ Zachowanie systemu jako interakcji pomiędzy tymi elementami
- ✓ Kompozycję elementów strukturalnych i behawioralnych w podsystemy

Co to jest architektura?

- ✓ Ogólna budowa systemu, w którego skład wchodzi oprogramowanie oraz infrastruktura techniczna (sprzęt)
- ✓ Łączy własności strukturalne oraz funkcjonalne
- ✓ Opisuje składowe, powiązania między nimi i kierunki przepływu danych



Architektura logiczna:

- ✓ Podział na logiczne elementy (moduły, pakiety, komponenty), które powinny współpracować w celu realizacji wymagań
- ✓ UML: komponenty + interfejsy



Jaka architektura?

Architektura fizyczna:

- ✓ Określenie fizycznych jednostek (maszyn) i metod komunikacji
- ✓ UML: węzeł

Architektura to fundament!

Określenie architektury, to podjęcie strategicznych decyzji, reguł i wzorców determinujących projektowanie i konstrukcję



Dobra architektura



Elastyczna

Spełnia bieżące i przyszłe wymagania

Umożliwia rozbudowę

Umożliwia ponowne użycie

Hermetyzuje zależnie części systemu



Modułowa

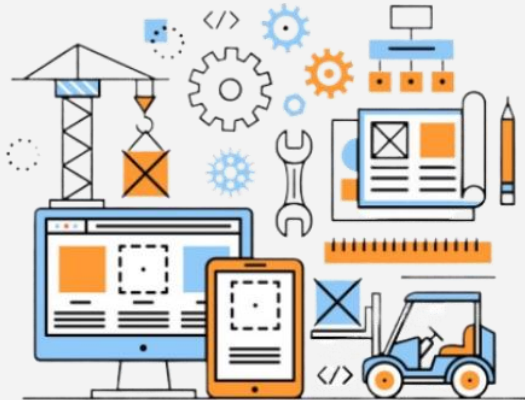
Umożliwia ponowne użycie i modyfikację modułów

Wybór z dostępnych na rynku komponentów

Inkrementacyjny rozwój oprogramowania

Moduł – komponent

Nietrywialna, prawie niezależna
i wymienna część systemu,
która realizuje zdefiniowaną
funkcjonalność.



Po co nam moduły?



Ponowne wykorzystanie

- ✓ Komponentów
- ✓ Architektury



Podstawa do zarządzania projektem

- ✓ Planowanie harmonogramu
- ✓ Podział prac



Kontrola

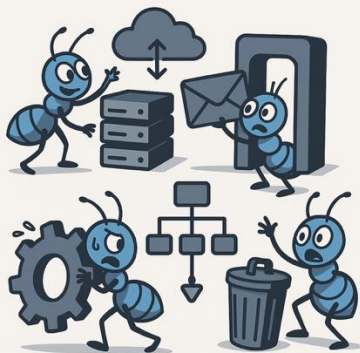
- ✓ Zarządzanie złożonym systemem
- ✓ Integracja



Nowe podejścia architektoniczne



Mikroserwisy, event-driven architecture



Serverless computing, API Gateway, Message Broker



Konteneryzacja: Docker, Kubernetes



Projektowanie skalowalnych i odpornych systemów

6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

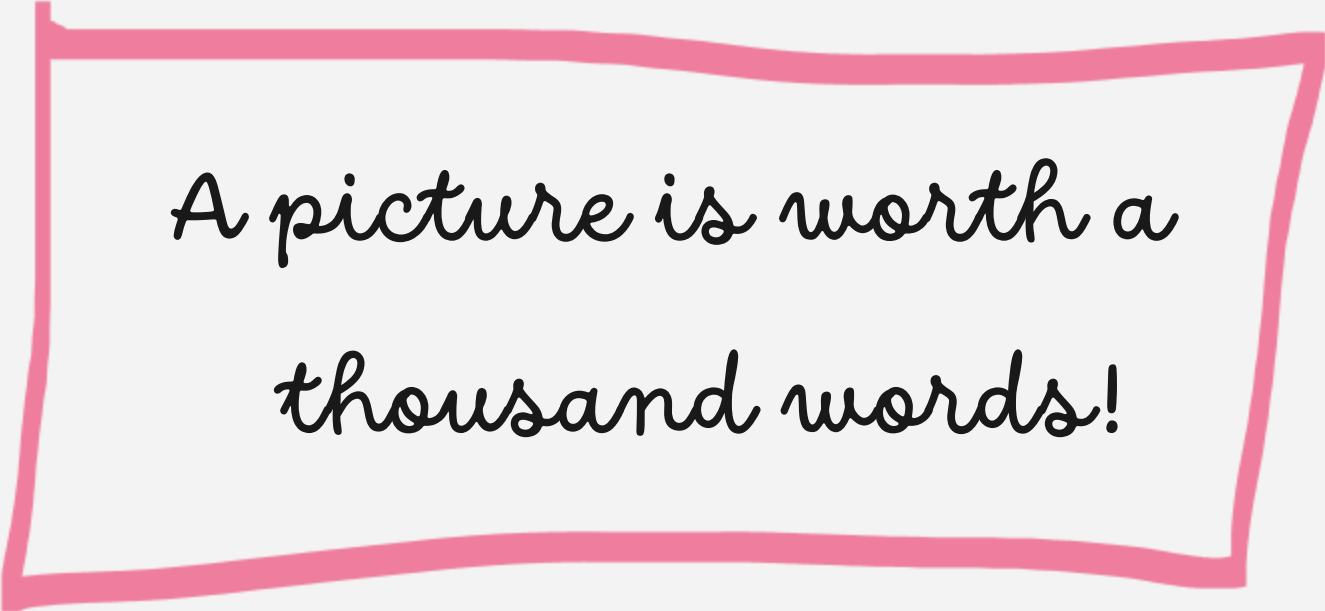
03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami





A picture is worth a
thousand words!

Korzyści z wizualizacji



- ✓ Uchwycenie struktury i zachowania
- ✓ Pokazanie zależności między elementami
- ✓ Utrzymanie spójnego projektu i implementacji
- ✓ Możliwe różne poziomy abstrakcji
- ✓ Promuje jeden wspólny język: UML



- ✓ Język do specyfikowania, wizualizowania, konstruowania i dokumentowania tworów oprogramowania i modelowania biznesu
- ✓ Reprezentuje zbiór najlepszych praktyk inżynierskich, które dowiodły przydatności przy modelowaniu dużych i skomplikowanych systemów

**UNIFIED
MODELING
LANGUAGE™**



- ✓ **UML 1.0 1997:** Grady Booch, Ivar Jacobson i James Rumbaugh (OMG)
- ✓ **UML 2.5.1 2017:** Object Management Group



<https://www.omg.org/about/>

The Object Management Group® Standards Development Organization (OMG® SDO) is an international (27 countries), membership-driven (230+ organizations) and not-for-profit consortium



STANDARDS

Known as an international standards development organization



SPECIFICATIONS

260+ OMG specifications are developed and maintained by our member.



MEMBERSHIP

220+ Member Organizations Worldwide



ISO

15 OMG specifications were adopted by ISO

MISSION STATEMENT

To generate technology standards (250+) that provide quantifiable real-world value to all vertical industries. That is why we are dedicated to bringing together our international [membership community](#) of end-users, researchers and vendors in academia government and industry to develop and revise our standards as technologies change over time.



<https://www.uml.org>

CERTIFICATION

TRAINING

RESOURCES

VENDORS

UNIFIED CELEBRATING 25 YEARS OF UML 1.1

One of the key figures in the software development community, Grady Booch describes the "grand journey" that led to the development of OMG UML standard.

WATCH VIDEO

25 YEARS
ANNIVERSARY

WHAT IS
UML?

UML
VENDOR

UML
SPECIFICATIONS

UML
CERTIFICATION

TRAINING
PAGE

**UNIFIED
MODELING
LANGUAGE™**



- ✓ IBM – IBM Rational
- ✓ Sparx Systems – Enterprise Architect
- ✓ Microsoft – Visual Studio oraz Team Foundation Server (TFS)
- ✓ Oracle – Oracle JDeveloper
- ✓ No Magic (teraz część Dassault Systèmes) – MagicDraw
- ✓ Altova – XMLSpy oraz Umodel
- ✓ Siemens – PLM (Product Lifecycle Management)

• Korzyści z UMLa •

- ✓ Wizualny język modelowania do budowy i wymiany modeli
- ✓ Niezależny od języków programowania i procesów tworzenia
- ✓ Udostępnia podstawy formalne do zrozumienia języka modelowania
- ✓ Zachęca do wzrostu rynku narzędzi OO
- ✓ Wspomaga wysoko-poziomowe rozwiązania
- ✓ Integruje najlepsze praktyki

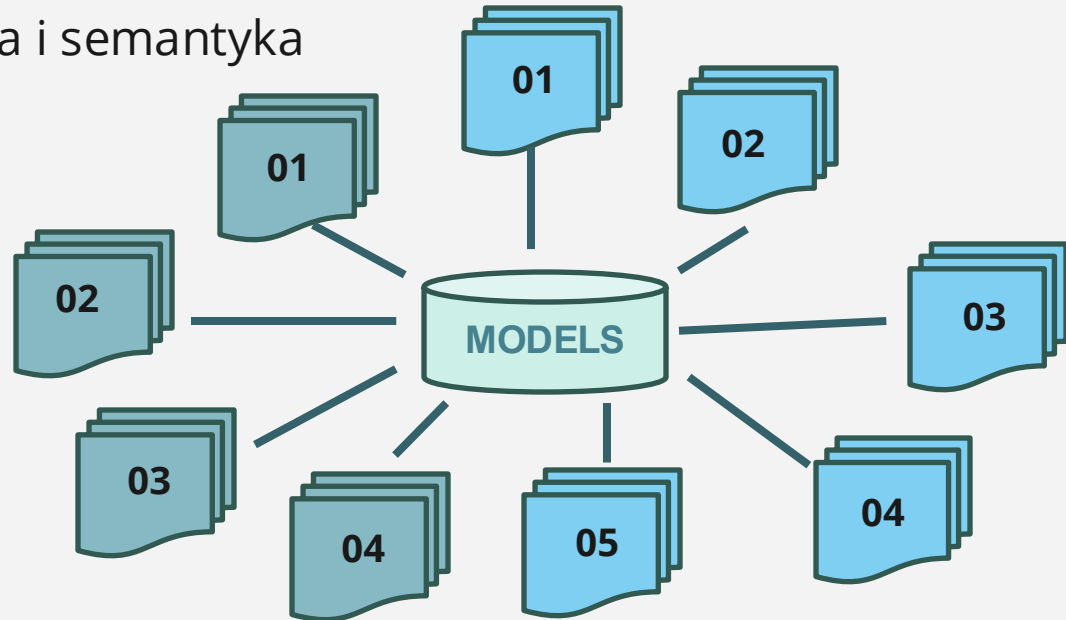


Wizualizacja & UML

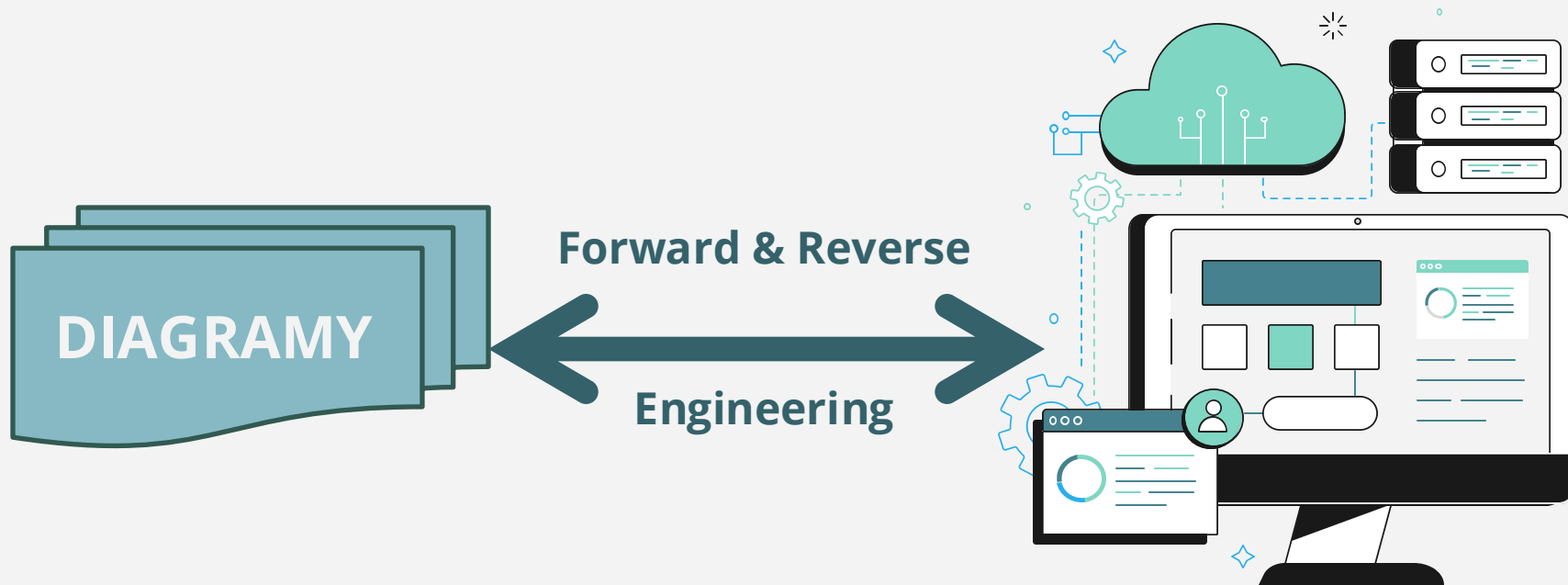
- ✓ Wiele punktów widzenia
- ✓ Precyzyjna syntaksa i semantyka

01 Use-Case Diagram
02 Class Diagram
03 Object Diagram
04 Component Diagram
05 Deployment Diagram

01 Sequence Diagram
02 Communication Diagram
03 State Machine Diagram
04 Activity Diagram



Wizualizacja & UML



• Krytyka UML



Wizualizacja w praktyce – nowoczesne modele

- ✓ C4 Model (Context → Container → Component → Code): Strukturyzator stworzone przez Simona Browna
- ✓ Model Driven Engineering (MDE) : Enterprise Architect, MagicDraw czy IBM Rhapsody
- ✓ PlantUML, Lucidchart, Draw.io, Diagrams.net
- ✓ Automatyczna dokumentacja modeli w CI/CD (C4-Builder)

6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami



Jakość



Charakterystyka wyniku produkcji oprogramowania określona na podstawie tego, czy produkt spełnia lub przewyższa **uzgodnione wymagania** oraz czy zostało ono wytworzone zgodnie z **ustalonym procesem**, co jest mierzone zgodnie z **ustalonymi kryteriami**.

Jakość



- ✓ Widoczna
- ✓ Ukryta
- ✓ Testy to za mało
- ✓ Oceny użytkownika
- ✓ Inspekcje modeli
- ✓ Inspekcje kodu
- ✓ Kontrola jakości w każdej iteracji!

Jakość



Wady oprogramowania są od 100 do 1000 razy droższe do zidentyfikowania i do naprawienia po wdrożeniu

- ✓ Koszt naprawy błędów
- ✓ Koszt straconych możliwości
- ✓ Koszt straconych klientów

Kompletne testowanie

FUNKCJONALNOŚĆ

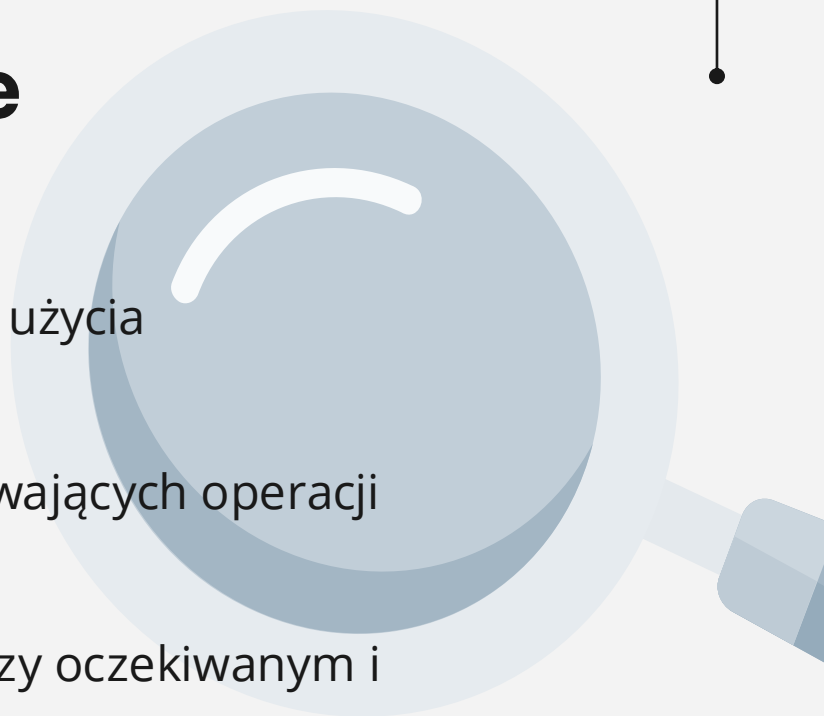
- ✓ Weryfikacja scenariuszy użycia

NIEZAWODNOŚĆ

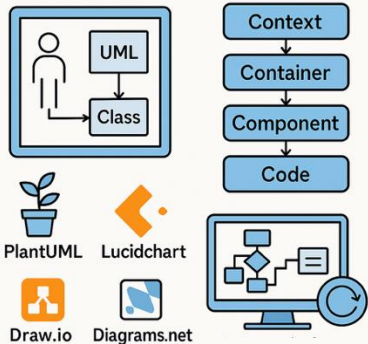
- ✓ Weryfikacja najdłużej trwających operacji

WYDAJNOŚĆ

- ✓ Weryfikacja działania przy oczekiwanym i najgorszym obciążeniu



Ciągła weryfikacja jakości



- ✓ Automatyczne testowanie jednostkowe i integracyjne
- ✓ Continuous Integration / Continuous Testing
- ✓ Monitorowanie jakości kodu: SonarQube, linters, code reviews

6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

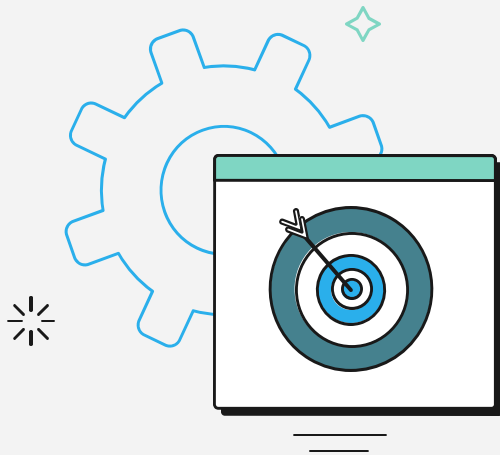
04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami

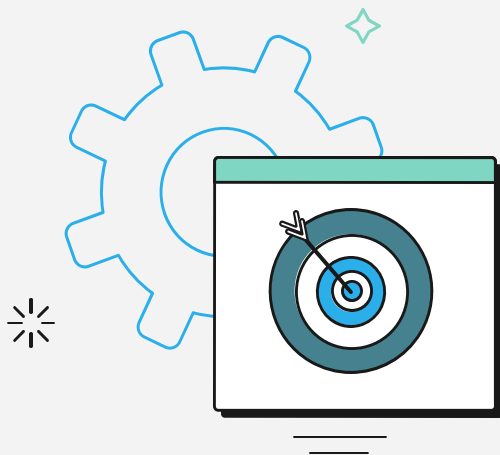


• Co chcemy kontrolować? •



- ✓ Zmiany
- ✓ Podział prac w zespole
- ✓ Integrację
- ✓ Równoległe prace

Istotne aspekty

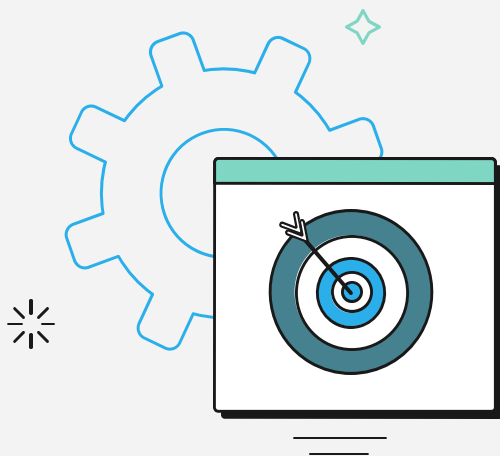


Zarządzanie zgłoszonymi zmianami
Formalny proces obsługi
Blokujemy „przeciekanie”
nowych wymagań

Raportowanie o stanie konfiguracji
Mierzenie rodzaju i liczby błędów

Zarządzanie konfiguracją
Narzędzia do wersjonowania i pracy
grupowej

Istotne aspekty



Śledzenie zmian

Formalny proces obsługi

Wybór wersji i produkowanie systemu

Automatyzacja kompilacji, testowania,
pakietowania i rozprowadzania

System kontroli wersji

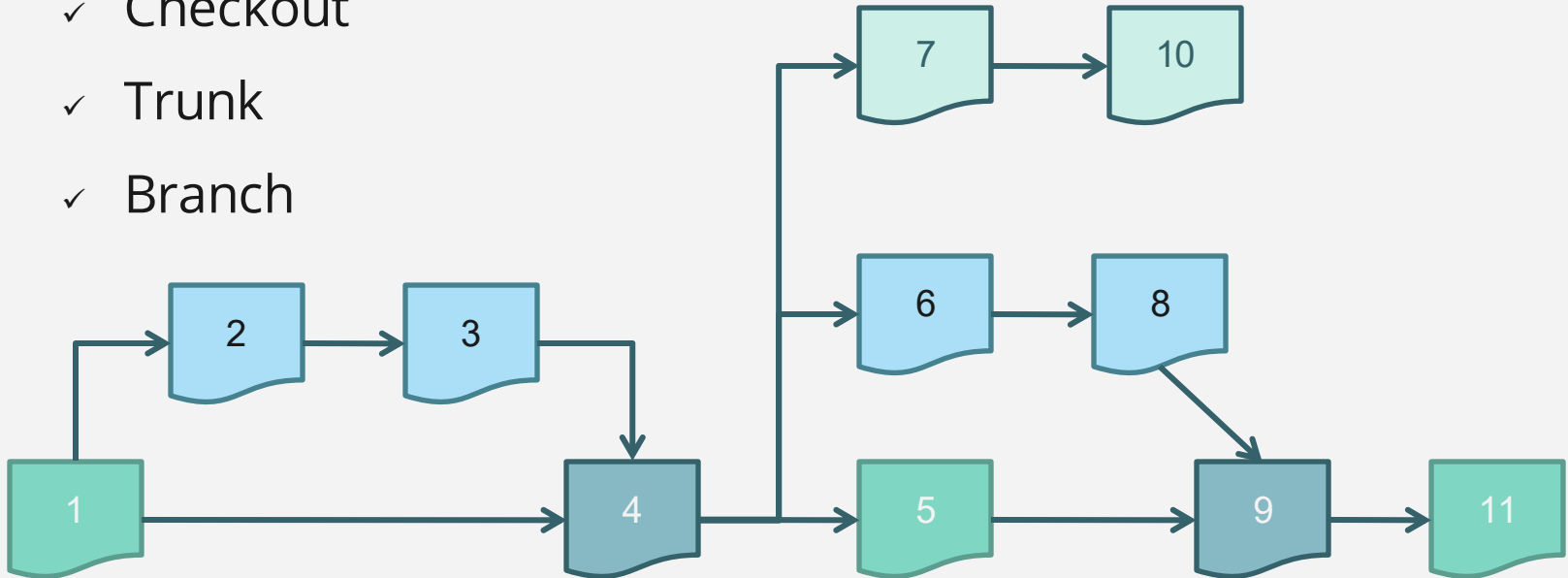
- ✓ Zarządzanie konfiguracją
- ✓ Zarządzanie katalogami, plikami, zmianami
- ✓ Architektura:
 - ✓ Scentralizowana klient-serwer
 - ✓ Rozproszona P2P
- ✓ Organizuje pracę w zespole



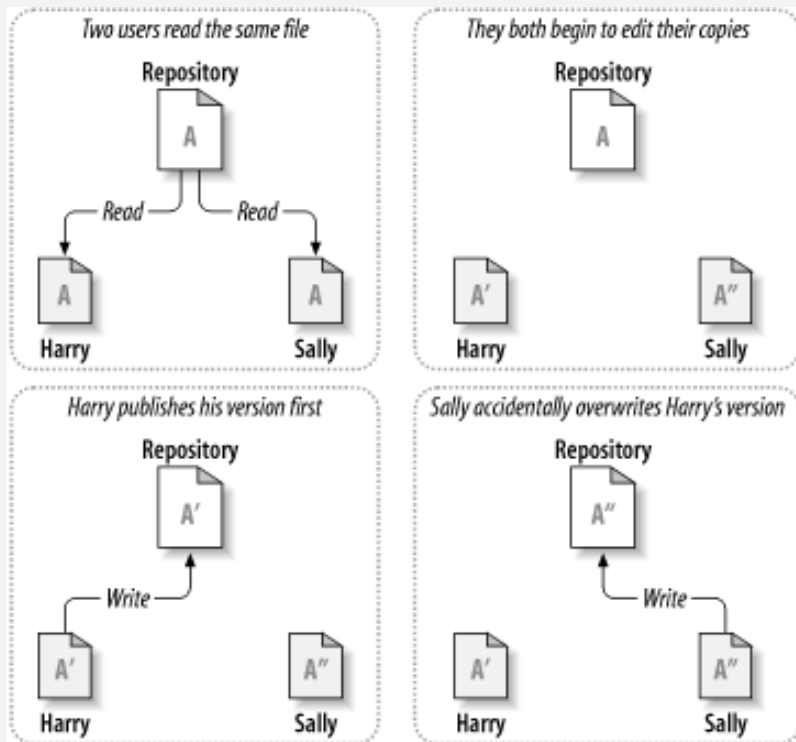
<https://www.accuwebhosting.com/blog/best-version-control-systems/>

System kontroli wersji

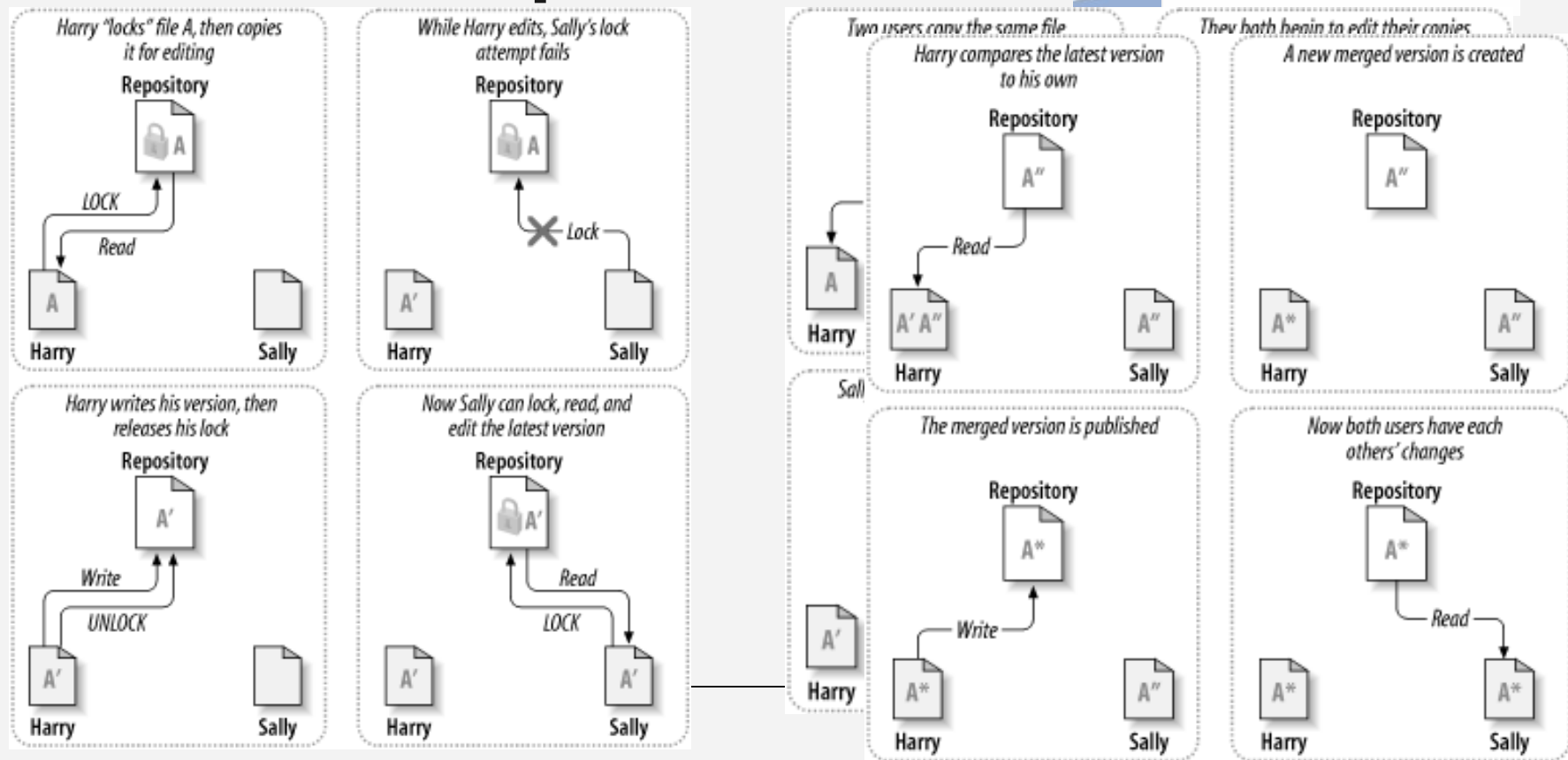
- ✓ Commit (chekin)
- ✓ Checkout
- ✓ Trunk
- ✓ Branch



Problem współdzielenia

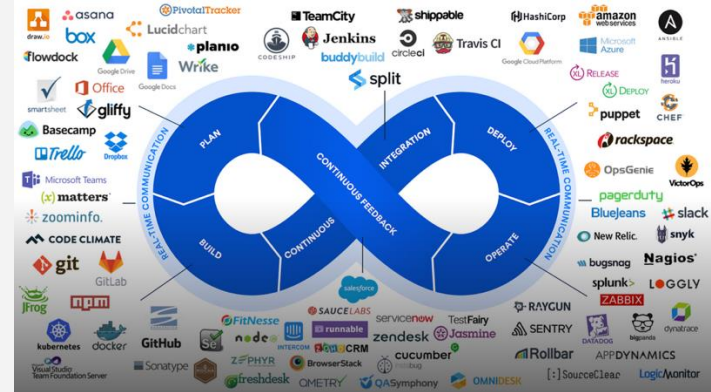


Problem współdzielenia



• Zarządzanie zmianami: nowoczesne praktyki •

- ✓ Systemy kontroli wersji: Git, GitHub, GitLab, Bitbucket
- ✓ Modele pracy: GitFlow, GitHub Flow
- ✓ Infrastructure as Code: Terraform, Ansible
- ✓ Automatyzacja wdrożeń i rollbacki
- ✓ Rejestrowanie zmian w środowisku DevOps



<https://www.unicodesystems.in/article/devops-revolutionizing-modern-software-development>

6 najlepszych praktyk IO

01 Podejście iteracyjne

02 Zarządzanie wymaganiami

03 Architektura modułowa i mikroserwisy

04 Wizualizacja modelu

05 Ciągła weryfikacja jakości

06 Zarządzanie zmianami

07 ?



Nie odkrywaj koła!

Wykorzystuj ponownie

- ✓ Klasy
- ✓ Wzorce projektowe
- ✓ Komponenty
- ✓ Interfejsy i wzorce analityczne
- ✓ Wzorce architektury
- ✓ Wzorce wymagań

Wyższy poziom to trudniej wykorzystać

Wyższy poziom to większe korzyści

07 ?





Koniec!